

Corrigé — Devoir 7 : Chaînes de caractères**Exercice 1 : Extraction d'initiales**

1. Ligne 2 : `init = identite[0]` affecte à `init` le premier caractère (rang 0) de la chaîne `identite`. Pour "Martin Durand-Dupont", `init` vaut 'M'.
2. La boucle parcourt tous les indices de 0 à `len(identite)-1`. À chaque indice i , si `identite[i]` est un espace **ou** un tiret, on ajoute `identite[i+1]` (la lettre qui suit) à `init`. Ce schème capture chaque début de mot ou de partie composée.
3. `.upper()` convertit toutes les lettres de `init` en majuscules, quelle que soit la casse saisie. C'est la première occurrence de la syntaxe `objet.méthode()` dans la séquence.
4. Résultats :
 - a) `initiales("martin durand")` → `init="m"+"d"="md"` → "MD"
 - b) `initiales("martin dupont durant")` → `init="m"+"d"+"d"="mdd"` → "MDD"
 - c) `initiales("martin dupont-durant")` → espace détecte "d", tiret détecte "d" → `init="m"+"d"+"d"="mdd"` → "MDD"
5. Sans `or identite[i] == "-"`, la condition ne détecte que les espaces. Pour "martin dupont-durant" : seul l'espace est détecté → `init="m"+"d"="md"` → "MD" au lieu de "MDD". Le tiret n'étant pas reconnu comme séparateur, "durant" n'est pas pris en compte.

Exercice 2 : L'escalier

1. Ligne 2 : `affiche = phrase[0]` initialise `affiche` avec le premier caractère de la phrase. C'est le point de départ de l'accumulation.
2. La boucle parcourt les indices de 1 à `len(phrase)-1`. À chaque tour, elle ajoute le caractère de rang i à `affiche` puis affiche le résultat. Chaque affichage est plus long que le précédent : on construit la phrase caractère par caractère.

3. Affichage complet de `escalier("Stylé")` :

```
S
St
Sty
Styl
Stylé
```

4. Avec `range(len(phrase))` la boucle démarre à $i = 0$. Au premier tour : `affiche = affiche + phrase[0] = "S"+"S" = "SS"` — doublement de la première lettre. La première ligne affichée devient "SS" au lieu de "S". Les suivantes sont décalées d'un caractère vers la droite.

Exercice 3 : Fonction essai

1. `len()` s'applique aux chaînes, pas aux entiers. `str(n)` convertit n en chaîne de caractères avant le calcul de longueur. Sans cette conversion, `len(123)` provoquerait une `TypeError`.

2. Résultats :

a) `essai(123)` : `str(123)="123"`, `len=3` → renvoie **3**

b) `essai("Bonjour")` : `str("Bonjour")="Bonjour"`, `len=7` → renvoie **7**

c) `essai(-596)` : `str(-596)="-596"`, le signe moins est un caractère, `len=4` → renvoie **4**

4. Cas limite révélateur.

3. Modification — renvoyer le chiffre des unités :

```
1 def chiffre_unites(n):
2     chiffres = str(n)
3     return chiffres[-1]      # dernier caractere = chiffre des unites
```

► `chiffres[-1]` accède au dernier caractère de la chaîne, c'est-à-dire le chiffre des unités. Exemples : `chiffre_unites(375)` → "5" ; `chiffre_unites(40)` → "0".

Exercice 4 : Approfondissement (Type et conversion)

1. `input()` renvoie une chaîne. `str("1234") = "1234"`. `chiffres[-2] = "3"` (avant-dernier caractère = chiffre des dizaines) → affiche **"3"**.

2. `str(n)` convertit n en chaîne de ses chiffres pour permettre l'indexation. Sans cette conversion, l'opérateur `[]` ne s'appliquerait pas.

3. Fonction `chiffre_dizaine` :

```
1 def chiffre_dizaine(n):
2     chiffres = str(n)
3     return chiffres[-2]      # avant-dernier = dizaines
```

4. Fonction `chiffre_unites` :

```
1 def chiffre_unites(n):
2     chiffres = str(n)
3     return chiffres[-1]      # dernier = unites
```

5. Pour $n = 7$: `str(7)="7"`, chaîne de longueur 1. `chiffres[-2]` demande l'avant-dernier caractère d'une chaîne de longueur 1 → `IndexError`. La fonction ne fonctionne que pour $n \geq 10$. C'est la limite silencieuse de cette approche.

► *Connexion math* : le chiffre des unités est aussi `n%10` (reste euclidien par 10). Ce sont deux chemins différents — l'arithmétique et la représentation textuelle — pour le même résultat.

Exercice 5 : Code César

1. a) Tableau complété :

Lettre en clair	Code ASCII	ASCII + 3	Lettre codée
E	69	72	H
C	67	70	F
S	83	86	V
A	65	68	D
R	82	85	U

b) Utiliser le code César revient à ajouter **3** au code ASCII de chaque lettre puis à appliquer `chr()` au résultat.

2. a) La boucle parcourt chaque caractère du message (converti en majuscules). Pour chaque caractère, elle calcule son code ASCII avec `ord()`, ajoute 3, reconvertit en lettre avec `chr()` et concatène le résultat à `coder`.

b) Codages :

► **CESAR** : C(67)→70=F, E(69)→72=H, S(83)→86=V, A(65)→68=D, R(82)→85=U → **FHVDU**

► **ROYAUME** : R→U, O→R, Y→\, A→D, U→X, M→P, E→H → **UR\DXPH** (problème sur Y !)

c) Problèmes :

► **X, Y, Z** : codes 88, 89, 90. En ajoutant 3 : 91, 92, 93 → hors alphabet (\,], ^).

► **Espace** : `ord(" ") = 32`. Espace +3 = 35 → `"#"` : l'espace est codé en `"#"` au lieu d'être conservé.

3. Version corrigée :

```

1 def code_cesar(message):
2     message = message.upper()
3     coder = ""
4     for caractere in message:
5         n = ord(caractere)
6         if n == 32:                # espace -> conserve
7             n_code = 32
8         elif n + 3 > 90:            # X, Y, Z -> rotation
9             n_code = n + 3 - 26
10        else:
11            n_code = n + 3
12            lettre_c = chr(n_code)
13            coder = coder + lettre_c
14    return coder

```

d) Codage de **ZYGOTE** :

► Z(90)+3 - 26 = 67 =C, Y(89)+3 - 26 = 66 =B, G(71)+3 = 74 =J, O(79)+3 = 82 =R, T(84)+3 = 87 =W, E(69)+3 = 72 =H → **CBJRW H**